Git und TEI gemeinsam untersuchen

Diagnose: Muster verstehen. Prävention: geeignete Prüfungen entwickeln. Evaluation: ihre Wirkung und Belastung überprüfen.

Eine Stelle genau untersuchen

Git blame, show und diff

Die Zeilenhistorie liefert einen Einstieg in die letzte zugeordnete Änderung. Frühere Stände und Unterschiede lassen sich anschließend gezielt lesen und mit Annotation, Prüfabzug und Korrekturvermerk verbinden.

Zu beachten: Keine automatische Ursachenermittlung. Gelöschte oder ersetzte Zeilen und frühere Veränderungen benötigen weitere Vergleiche. Der zugeordnete Name ist kein Beweis individueller Schuld.

git blame ↗ · git show ↗ · git diff ↗

Erster Überblick

Git Quick Stats

Zeigt Statistiken über Commits, Dateien und Aktivität. Samir prüft damit zunächst, welche Bereiche und Zeiträume einen genaueren Blick verdienen.

Zu beachten: Keine TEI-Semantik. Importe, Bots, Umbenennungen und Massenformatierungen können das Bild prägen; Statistiken erklären ihre Ursachen nicht.

Projekt und Anwendungsbeispiele ↗

Gezielter Bericht

Gitinspector

Erstellt historische Repository-Berichte und kann auf ausgewählte Dateitypen eingegrenzt werden. Für TEI-Dateien muss Samir XML ausdrücklich in den Dateifilter aufnehmen.

Zu beachten: Standardberichte sind personenorientiert, XML ist im voreingestellten Dateifilter nicht enthalten. Eine XML-Ausgabedatei des Berichts ist keine Analyse der TEI-Bedeutung. Auswahl und Zugriff vorab klären.

Projekt und Dokumentation ↗

Im Team ansehen

Gource

Visualisiert die Entwicklung eines Repositories als bewegten Dateibaum. Für einen Workshop kann das Wachstum und Wechsel der Bearbeitung anschaulich werden.

Zu beachten: Eine anschauliche Animation ist kein Qualitätsnachweis. Namen und Aktivität können sichtbar werden; auch die gemeinsame Vorführung benötigt einen geklärten Umfang.

Projekt, Beispiele und Einrichtung ↗

Git und TEI verbinden

PyDriller

Eine Python-Bibliothek zum Durchlaufen von Commits und geänderten Dateien. Samir kann frühere und spätere Dateiinhalte mit eigenen XML-Prüfungen verbinden.

Zu beachten: Eigene Skripte und fachliche Definitionen nötig. Eingebaute Code-/Methodenmetriken sind keine TEI-Qualitätsmessung. Fehlende Inhalte, Umbenennungen und Merge-Strategie berücksichtigen.

Dokumentation zu geänderten Dateien ↗ · Projekt und Einstieg ↗

Die XML-Seite prüfen

Jing, Schematron, XPath und XSLT

Jing prüft gegen ein RELAX-NG-Schema. Schematron formuliert zusätzliche Regeln. XPath wählt Elemente und Attribute aus; XSLT kann strukturierte Vergleiche, Berichte und Ausgaben erzeugen.

Zu beachten: Das sind unterschiedliche Werkzeuge und Sprachen. Regelversion, Kontext und Prüflauf werden dokumentiert. Ein formal gültiger Bezug kann auf die fachlich falsche Stelle zeigen.

Jing ↗ · Schematron ↗ · XPath ↗

Muster ergänzend erkunden

CodeScene / Code Maat

Hotspots beziehungsweise gemeinsam geänderte Dateien liefern Suchrichtungen. Samir vergleicht diese mit den fachlich eingeordneten Korrekturfällen.

Zu beachten: Code-Health-Bewertungen nicht auf TEI übertragen. Ein häufig geänderter Bereich kann intensive wissenschaftliche Arbeit statt schlechter Qualität zeigen.

CodeScene: Produkt und Tour ↗ · Code Maat ↗

Bei größerem Bedarf

GrimoireLab

Kann Daten aus Git, Vorgängen und Review-Systemen zusammenführen und als Dashboards erschließen. Interessant, wenn ein Pilot einen begründeten dauerhaften Bedarf ergibt.

Zu beachten: Mehr Einrichtung und Pflege; die verknüpften Daten ermöglichen auch tiefere personenbezogene Einsichten. Nicht vorsorglich alle verfügbaren Quellen sammeln.

Projekt, unterstützte Quellen und Einstieg ↗

Ein begrenzter Pilot: verstehen, verbessern, überprüfen

1. Inventur und klare Frage

Ein bis drei geeignete Repositories und vergleichbare Zeiträume wählen. Mit einem kleinen Überblick beginnen und Importe, Bots, Umbenennungen sowie geänderte Commit-Praxis kennzeichnen. Vor der Auswertung Zweck, Datenumfang und Zugriff klären.

2. Git- und TEI-Befunde verbinden

Dieselben Teilbestände gegen das passende Schema und Projektregeln prüfen. Historische Stände mit PyDriller und XML-Auswertungen untersuchen. Bekannte Problemfälle mit vergleichbaren stabilen Bereichen abgleichen; fachliche Entscheidungen von Fehlerkorrekturen unterscheiden.

3. Maßnahmen und Frühindikatoren erproben

Drei bis fünf Auffälligkeiten mit alternativen Erklärungen und priorisierten Maßnahmen festhalten. Verbesserungen gezielt testen. Einen möglichen Frühindikator erst an späteren, nicht für seine Entwicklung verwendeten Zeiträumen prüfen; Fehlalarme und übersehene Probleme erfassen. Kein pauschaler Edition-Health-Score.

Die Werkzeuge werden hier erklärt und verlinkt, nicht auf Projektdaten ausgeführt. Datenquellen, historische Personenbezüge, Zugriff und Einspruchsweg werden vor dem Pilot geklärt.